Project.js

```
const mongoose = require("mongoose");

const ProjectSchema = new mongoose.Schema(
  {
    projectName: {
      type: String,
      required: true
    },

    projectType: {
      type: String,
      default: "House Construction"
    },

    location: {
      type: String,
      required: true
    },

    contractor: {
      type: String,
      required: true
    },

    budget: {
      type: Number,
      required: true
    },

    budgetUsed: {
      type: Number,
      default: 0
    },

    progress: {
      type: Number,
      default: 0
    },

    status: {
      type: String,
      default: "Ongoing"
```

```
    },

    startDate: {
      type: Date,
      default: null
    },

    expectedCompletion: {
      type: Date,
      default: null
    },

    description: {
      type: String,
      default: ""
    }
  },
  {
    timestamps: true
  }
);

module.exports = mongoose.model("Project", ProjectSchema);
```

Projects.js

```
const express = require("express");
const router = express.Router();

const Project = require("../models/Project");

// ================================
// GET ALL PROJECTS
// ================================

router.get("/", async (req, res) => {
  try {
    const projects = await Project.find().sort({ createdAt: -1 });

    res.status(200).json(projects);

  } catch (error) {
    console.error("GET PROJECTS ERROR:", error);

    res.status(500).json({
```

```
      success: false,
      message: error.message
    });
  }
});

// ================================
// CREATE PROJECT
// ================================

router.post("/", async (req, res) => {
  try {
    console.log("==================================");
    console.log("NEW PROJECT REQUEST");
    console.log("BODY:", req.body);
    console.log("==================================");

    const newProject = new Project({
      projectName: req.body.projectName,
      projectType: req.body.projectType || "House Construction",
      location: req.body.location,
      contractor: req.body.contractor,
      budget: Number(req.body.budget),
      budgetUsed: Number(req.body.budgetUsed) || 0,
      progress: Number(req.body.progress) || 0,
      status: req.body.status || "Ongoing",
      startDate: req.body.startDate || null,
      expectedCompletion: req.body.expectedCompletion || null,
      description: req.body.description || ""
    });

    const savedProject = await newProject.save();

    console.log("PROJECT SAVED SUCCESSFULLY:");
    console.log(savedProject);

    res.status(201).json({
      success: true,
      message: "Project created successfully",
      project: savedProject
    });

  } catch (error) {
    console.error("==================================");
    console.error("CREATE PROJECT ERROR");
```

```
    console.error(error);
    console.error("==================================");

    res.status(500).json({
      success: false,
      message: error.message
    });
  }
});

module.exports = router;
```

Server.js

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");
require("dotenv").config();

const app = express();

// ================================
// MIDDLEWARE
// ================================

app.use(cors());
app.use(express.json());

// ================================
// PORT
// ================================

const PORT = process.env.PORT || 5000;

// ================================
// ROUTES
// ================================

const userRoutes = require("./routes/users");
const projectRoutes = require("./routes/projects");

app.use("/api/users", userRoutes);
app.use("/api/projects", projectRoutes);
```

```
// ================================
// HOME ROUTE
// ================================

app.get("/", (req, res) => {
  res.send("Construction X Backend Working");
});

// ================================
// MONGODB CONNECTION
// ================================

console.log("Mongo URI:", process.env.MONGO_URI);

mongoose
  .connect(process.env.MONGO_URI, {
    serverSelectionTimeoutMS: 5000
  })
  .then(() => {
    console.log("==================================");
    console.log("MONGODB CONNECTED SUCCESSFULLY");
    console.log("DATABASE:", mongoose.connection.name);
    console.log("HOST:", mongoose.connection.host);
    console.log("==================================");

    // ================================
    // START SERVER ONLY AFTER DATABASE
    // CONNECTS SUCCESSFULLY
    // ================================

    app.listen(PORT, () => {
      console.log("Construction X Server Started");
      console.log(`Server running on port ${PORT}`);
    });
  })
  .catch((error) => {
    console.error("==================================");
    console.error("MONGODB CONNECTION FAILED");
    console.error(error.message);
    console.error("==================================");

    process.exit(1);
  });
```

PROJECTPAGE.JS

```
import React, { useEffect, useState } from "react";
import axios from "axios";

const API_URL = "http://localhost:5000/api/projects";

const initialFormData = {
  projectName: "",
  projectType: "House Construction",
  location: "",
  contractor: "",
  budget: "",
  budgetUsed: "0",
  progress: 0,
  status: "Ongoing",
  startDate: "",
  expectedCompletion: "",
  description: ""
};

export default function ProjectsPage() {
  const [projects, setProjects] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState("");

  const [showModal, setShowModal] = useState(false);
  const [searchTerm, setSearchTerm] = useState("");
  const [statusFilter, setStatusFilter] = useState("All");
  const [submitting, setSubmitting] = useState(false);

  const [formData, setFormData] = useState(initialFormData);

  // ========================
  // FETCH PROJECTS
  // ========================

  const fetchProjects = async () => {
    try {
      setLoading(true);
      setError("");

      const response = await axios.get(API_URL);
```

```
    // Supports:
    // 1. [project1, project2]
    // 2. { projects: [...] }
    const data = response.data;

    if (Array.isArray(data)) {
      setProjects(data);
    } else if (Array.isArray(data.projects)) {
      setProjects(data.projects);
    } else {
      setProjects([]);
    }
  } catch (error) {
    console.error("FETCH PROJECTS ERROR:", error);

    setError(
      error.response?.data?.message ||
        error.response?.data?.error ||
        "Unable to fetch projects"
    );
  } finally {
    setLoading(false);
  }
};

useEffect(() => {
  fetchProjects();
}, []);

// ==========================
// HANDLE INPUT
// ==========================

const handleChange = (e) => {
  const { name, value } = e.target;

  setFormData((previousData) => ({
    ...previousData,
    [name]: value
  }));
};

// ==========================
// ADD PROJECT
// ==========================
```

```
const handleSubmit = async (e) => {
  e.preventDefault();

  // Basic validation
  if (
    !formData.projectName.trim() ||
    !formData.location.trim() ||
    !formData.contractor.trim() ||
    !formData.budget
  ) {
    alert("Please fill all required fields.");
    return;
  }

  const budget = Number(formData.budget);
  const budgetUsed = Number(formData.budgetUsed || 0);
  const progress = Number(formData.progress || 0);

  if (budget <= 0) {
    alert("Total budget must be greater than ₹0.");
    return;
  }

  if (budgetUsed < 0) {
    alert("Budget used cannot be negative.");
    return;
  }

  if (budgetUsed > budget) {
    alert("Budget used cannot be greater than the total budget.");
    return;
  }

  if (progress < 0 || progress > 100) {
    alert("Progress must be between 0 and 100.");
    return;
  }

  try {
    setSubmitting(true);

    // Send clean data to backend
    const projectData = {
      projectName: formData.projectName.trim(),
```

```
      projectType: formData.projectType,
      location: formData.location.trim(),
      contractor: formData.contractor.trim(),
      budget: budget,
      budgetUsed: budgetUsed,
      progress: progress,
      status: formData.status,
      startDate: formData.startDate || null,
      expectedCompletion: formData.expectedCompletion || null,
      description: formData.description.trim()
    };

    console.log("Sending project:", projectData);

    const response = await axios.post(API_URL, projectData, {
      headers: {
        "Content-Type": "application/json"
      }
    });

    console.log("Server response:", response.data);

    /*
      Backend might return either:

      { project: {...} }

      OR

      {...project}
    */

    const createdProject =
      response.data?.project || response.data?.data || response.data;

    if (!createdProject || !createdProject._id) {
      console.error(
        "Unexpected server response:",
        response.data
      );

      throw new Error("Invalid response received from server.");
    }

    // Add new project to top of list
```

```
      setProjects((previousProjects) => [
        createdProject,
        ...previousProjects
      ]);

      // Reset form
      setFormData(initialFormData);

      // Close modal
      setShowModal(false);

      alert("Project created successfully!");
    } catch (error) {
  console.error("================================");
  console.error("PROJECT CREATION ERROR");
  console.error("================================");

  console.error("Full error:", error);

  if (error.response) {
    console.error("Status:", error.response.status);
    console.error("Backend response:", error.response.data);

    const actualError =
      error.response.data?.error ||
      error.response.data?.message ||
      "Unknown server error";

    alert(
      "Unable to create project.\n\n" +
      "Backend error:\n" +
      actualError
    );

  } else if (error.request) {

    console.error("No response received:", error.request);

    alert(
      "Unable to create project.\n\n" +
      "Cannot connect to backend on port 5000."
    );

  } else {
```

```
    console.error("Request error:", error.message);

    alert(
      "Unable to create project.\n\n" +
      error.message
    );
  }

} finally {
  setSubmitting(false);
}
  };

  // ==========================
  // FORMAT CURRENCY
  // ==========================

  const formatCurrency = (amount) => {
    if (amount === undefined || amount === null || amount === "") {
      return "₹0";
    }

    return new Intl.NumberFormat("en-IN", {
      style: "currency",
      currency: "INR",
      maximumFractionDigits: 0
    }).format(Number(amount));
  };

  // ==========================
  // FILTER PROJECTS
  // ==========================

  const filteredProjects = projects.filter((project) => {
    const search = searchTerm.toLowerCase();

    const matchesSearch =
      project.projectName
        ?.toLowerCase()
        .includes(search) ||
      project.location
        ?.toLowerCase()
        .includes(search) ||
      project.contractor
        ?.toLowerCase()
```

```
      .includes(search) ||
    project.projectType
      ?.toLowerCase()
      .includes(search);

  const matchesStatus =
    statusFilter === "All" ||
    project.status === statusFilter;

  return matchesSearch && matchesStatus;
});

// ==========================
// LOADING
// ==========================

if (loading) {
  return (
    <div className="projects-page">
      <div className="projects-loading">
        <div className="loading-spinner"></div>

        <h2>Loading projects...</h2>

        <p>
          Preparing your construction dashboard
        </p>
      </div>
    </div>
  );
}

// ==========================
// ERROR
// ==========================

if (error) {
  return (
    <div className="projects-page">
      <div className="projects-error">
        <div className="error-icon">!</div>

        <h2>{error}</h2>

        <button
```

```
            onClick={fetchProjects}
            className="retry-btn"
          >
            Try Again
          </button>
        </div>
      </div>
    );
  }

  // ==========================
  // PAGE
  // ==========================

  return (
    <div className="projects-page">

      {/* ==========================
          HEADER
      ========================== */}

      <div className="projects-header">

        <div>
          <p className="projects-eyebrow">
            CONSTRUCTION MANAGEMENT
          </p>

          <h1>My Projects</h1>

          <p className="projects-subtitle">
            Manage, monitor and track your construction
            projects in one place.
          </p>
        </div>

        <button
          className="add-project-btn"
          onClick={() => setShowModal(true)}
        >
          <span>+</span>
          Add Project
        </button>

      </div>
```

```jsx
{/* ========================
    STATISTICS
======================== */}

<div className="project-stats">

  <div className="stat-card">
    <div className="stat-icon blue-icon">

    </div>

    <div>
      <span>Total Projects</span>
      <strong>{projects.length}</strong>
    </div>
  </div>

  <div className="stat-card">
    <div className="stat-icon orange-icon">

    </div>

    <div>
      <span>Ongoing</span>

      <strong>
        {
          projects.filter(
            (p) => p.status === "Ongoing"
          ).length
        }
      </strong>
    </div>
  </div>

  <div className="stat-card">
    <div className="stat-icon green-icon">
     ✓
    </div>

    <div>
      <span>Completed</span>

      <strong>
```

```
            {
              projects.filter(
                (p) => p.status === "Completed"
              ).length
            }
          </strong>
        </div>
      </div>

      <div className="stat-card">
        <div className="stat-icon purple-icon">
          ₹
        </div>

        <div>
          <span>Total Budget</span>

          <strong>
            {formatCurrency(
              projects.reduce(
                (total, project) =>
                  total +
                  Number(project.budget || 0),
                0
              )
            )}
          </strong>
        </div>
      </div>

    </div>

    {/* =========================
        SEARCH + FILTER
    ========================= */}

    <div className="project-toolbar">

      <div className="search-box">

        <span> </span>

        <input
          type="text"
          placeholder="Search projects, location or contractor..."
```

```
          value={searchTerm}
          onChange={(e) =>
            setSearchTerm(e.target.value)
          }
        />

      </div>

      <div className="filter-buttons">

        {[
          "All",
          "Ongoing",
          "Completed",
          "Planning"
        ].map((status) => (

          <button
            key={status}
            className={
              statusFilter === status
                ? "filter-btn active"
                : "filter-btn"
            }
            onClick={() =>
              setStatusFilter(status)
            }
          >
            {status}
          </button>

        ))}

      </div>

    </div>

    {/* ========================
        SECTION TITLE
    ======================== */}

    <div className="project-section-header">

      <div>
        <h2>Construction Projects</h2>
```

```
          <p>
            {filteredProjects.length} project
            {filteredProjects.length !== 1
              ? "s"
              : ""}{" "}
            found
          </p>
        </div>

      </div>

      {/* ========================
          EMPTY STATE
      ======================== */}

      {filteredProjects.length === 0 ? (

        <div className="empty-projects">

          <div className="empty-project-icon">

          </div>

          <h2>
            {projects.length === 0
              ? "No projects yet"
              : "No projects found"}
          </h2>

          <p>
            {projects.length === 0
              ? "Start managing your construction work by adding your first
project."
              : "Try changing your search or status filter."}
          </p>

          {projects.length === 0 && (
            <button
              className="add-project-btn"
              onClick={() => setShowModal(true)}
            >
              <span>+</span>
              Add Your First Project
            </button>
```

```
        )}

      </div>

    ) : (

      /* ========================
         PROJECT CARDS
      ======================== */

      <div className="project-grid">

        {filteredProjects.map((project) => {

          const progress = Math.min(
            100,
            Math.max(
              0,
              Number(project.progress || 0)
            )
          );

          const budget =
            Number(project.budget || 0);

          const budgetUsed =
            Number(project.budgetUsed || 0);

          const remainingBudget =
            Math.max(
              0,
              budget - budgetUsed
            );

          return (

            <div
              className="project-card"
              key={project._id}
            >

              {/* Card Top */}

              <div className="project-card-top">
```

```
            <div className="project-type-icon">

              {project.projectType ===
              "House Construction"
                ? ""
                : project.projectType ===
                  "Commercial Building"
                ? ""
                : project.projectType ===
                  "Temple / Religious"
                ? ""
                : project.projectType ===
                  "Road Construction"
                ? ""
                : project.projectType ===
                  "Renovation"
                ? ""
                : ""}

            </div>

            <span
              className={`status-badge ${
                project.status ===
                "Completed"
                  ? "completed"
                  : project.status ===
                    "Planning"
                  ? "planning"
                  : "ongoing"
              }`}
            >
              {project.status}
            </span>

          </div>

          {/* Project Name */}

          <h2 className="project-title">
            {project.projectName}
          </h2>

          <p className="project-type">
            {project.projectType ||
```

```
    "Construction Project"}
</p>

{/* Location */}

<div className="project-info">
  <span></span>
  <span>
    {project.location}
  </span>
</div>

{/* Contractor */}

<div className="project-info">
  <span></span>
  <span>
    {project.contractor}
  </span>
</div>

{/* Progress */}

<div className="progress-section">

  <div className="progress-header">

    <span>
      Project Progress
    </span>

    <strong>
      {progress}%
    </strong>

  </div>

  <div className="progress-bar">

    <div
      className="progress-fill"
      style={{
        width: `${progress}%`
      }}
    ></div>
```

```
      </div>

    </div>

    {/* Budget */}

    <div className="budget-section">

      <div>
        <span>Total Budget</span>

        <strong>
          {formatCurrency(budget)}
        </strong>
      </div>

      <div>
        <span>Used</span>

        <strong>
          {formatCurrency(budgetUsed)}
        </strong>
      </div>

    </div>

    {/* Remaining */}

    <div className="completion-date">

      <span>
          Remaining Budget
      </span>

      <strong>
        {formatCurrency(
          remainingBudget
        )}
      </strong>

    </div>

    {/* Completion Date */}
```

```
                {project.expectedCompletion && (

                  <div className="completion-date">

                    <span>
                       Expected completion
                    </span>

                    <strong>
                      {new Date(
                        project.expectedCompletion
                      ).toLocaleDateString(
                        "en-IN"
                      )}
                    </strong>

                  </div>

                )}

                {/* Footer */}

                <div className="project-card-footer">

                  <button
                    className="view-project-btn"
                    onClick={() =>
                      alert(
                        `Project: ${project.projectName}\n\nLocation:
${project.location}\nContractor: ${project.contractor}\nProgress: ${progress}%`
                      )
                    }
                  >
                    View Details
                    <span>→</span>
                  </button>

                </div>

              </div>

            );
          })}

        </div>
```

```
)}

{/* ========================
    ADD PROJECT MODAL
======================== */}

{showModal && (

  <div
    className="modal-overlay"
    onClick={() =>
      !submitting &&
      setShowModal(false)
    }
  >

    <div
      className="project-modal"
      onClick={(e) =>
        e.stopPropagation()
      }
    >

      {/* Modal Header */}

      <div className="modal-header">

        <div>

          <p className="modal-eyebrow">
            NEW PROJECT
          </p>

          <h2>
            Add New Project
          </h2>

          <p>
            Enter the details of your
            construction project.
          </p>

        </div>
```

```
            <button
              className="close-modal"
              onClick={() =>
                !submitting &&
                setShowModal(false)
              }
              disabled={submitting}
            >
              ×
            </button>

          </div>

          {/* Form */}

          <form onSubmit={handleSubmit}>

            {/* Row 1 */}

            <div className="form-row">

              <div className="project-form-group">

                <label>
                  Project Name *
                </label>

                <input
                  type="text"
                  name="projectName"
                  placeholder="e.g. Modern Family House"
                  value={
                    formData.projectName
                  }
                  onChange={handleChange}
                  required
                />

              </div>

              <div className="project-form-group">

                <label>
                  Project Type *
                </label>
```

```
          <select
            name="projectType"
            value={
              formData.projectType
            }
            onChange={handleChange}
          >
            <option>
              House Construction
            </option>

            <option>
              Commercial Building
            </option>

            <option>
              Temple / Religious
            </option>

            <option>
              Road Construction
            </option>

            <option>
              Renovation
            </option>

            <option>
              Other
            </option>

          </select>

        </div>

      </div>

      {/* Row 2 */}

      <div className="form-row">

        <div className="project-form-group">

          <label>
```

```
              Location *
            </label>

            <input
              type="text"
              name="location"
              placeholder="e.g. Mumbai, Maharashtra"
              value={
                formData.location
              }
              onChange={handleChange}
              required
            />

          </div>

          <div className="project-form-group">

            <label>
              Contractor *
            </label>

            <input
              type="text"
              name="contractor"
              placeholder="Contractor name"
              value={
                formData.contractor
              }
              onChange={handleChange}
              required
            />

          </div>

        </div>

        {/* Row 3 */}

        <div className="form-row">

          <div className="project-form-group">

            <label>
              Total Budget (₹) *
```

```
            </label>

            <input
              type="number"
              name="budget"
              placeholder="e.g. 2500000"
              value={formData.budget}
              onChange={handleChange}
              min="1"
              required
            />

          </div>

          <div className="project-form-group">

            <label>
              Budget Used (₹)
            </label>

            <input
              type="number"
              name="budgetUsed"
              placeholder="e.g. 500000"
              value={
                formData.budgetUsed
              }
              onChange={handleChange}
              min="0"
            />

          </div>

        </div>

        {/* Row 4 */}

        <div className="form-row">

          <div className="project-form-group">

            <label>
              Start Date
            </label>
```

```
            <input
              type="date"
              name="startDate"
              value={
                formData.startDate
              }
              onChange={handleChange}
            />

          </div>

          <div className="project-form-group">

            <label>
              Expected Completion
            </label>

            <input
              type="date"
              name="expectedCompletion"
              value={
                formData.expectedCompletion
              }
              onChange={handleChange}
            />

          </div>

        </div>

        {/* Row 5 */}

        <div className="form-row">

          <div className="project-form-group">

            <label>
              Progress (%)
            </label>

            <input
              type="number"
              name="progress"
              min="0"
              max="100"
```

```
            value={
              formData.progress
            }
            onChange={handleChange}
          />

        </div>

        <div className="project-form-group">

          <label>
            Status
          </label>

          <select
            name="status"
            value={
              formData.status
            }
            onChange={handleChange}
          >

            <option>
              Planning
            </option>

            <option>
              Ongoing
            </option>

            <option>
              Completed
            </option>

          </select>

        </div>

      </div>

      {/* Description */}

      <div className="project-form-group">

        <label>
```

```
            Description
          </label>

          <textarea
            name="description"
            placeholder="Briefly describe the project..."
            value={
              formData.description
            }
            onChange={handleChange}
            rows="3"
          ></textarea>

        </div>

        {/* Actions */}

        <div className="modal-actions">

          <button
            type="button"
            className="cancel-btn"
            onClick={() =>
              setShowModal(false)
            }
            disabled={submitting}
          >
            Cancel
          </button>

          <button
            type="submit"
            className="create-project-btn"
            disabled={submitting}
          >

            {submitting
              ? "Creating..."
              : "Create Project"}

          </button>

        </div>

      </form>
```

```
          </div>

        </div>

      )}

    </div>
  );
}
```

Construction X

Home Projects User Profile

BUILD • MANAGE • CONNECT

Welcome to Construction X

A smart platform designed to connect customers and contractors and simplify construction project management.

Login View Profile

Why Construction X?

Project Management

Manage your construction projects efficiently from one platform.

Find Contractors

Connect with contractors for your construction requirements.

Track Progress

Monitor project activities and keep track of your construction work.

© 2026 Construction X. All Rights Reserved.

CONSTRUCTION MANAGEMENT

My Projects

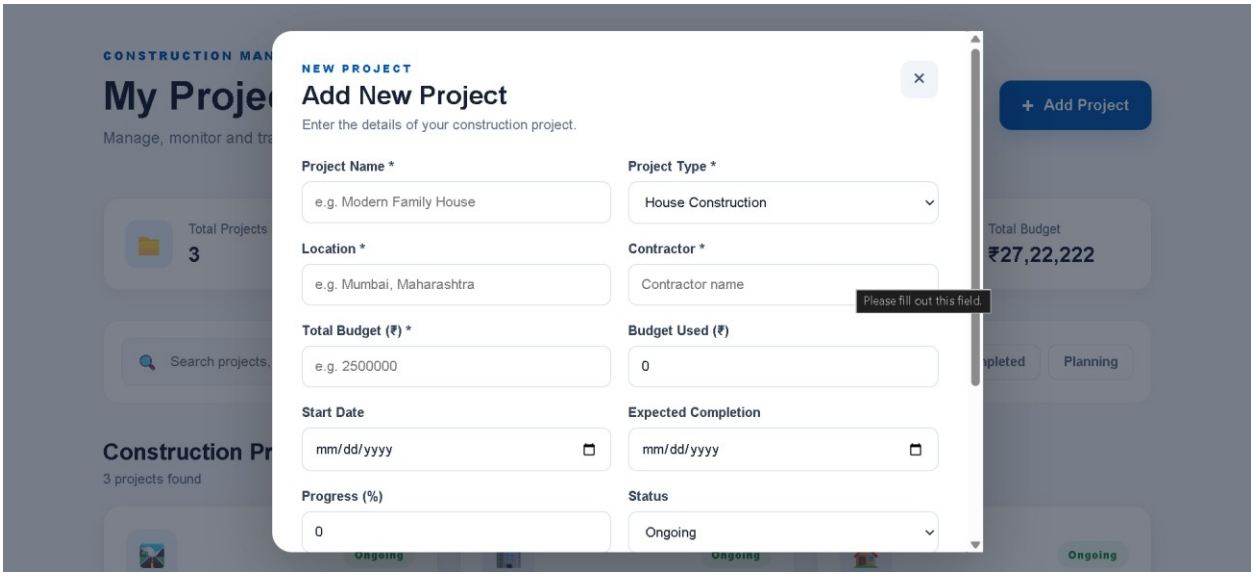
Manage, monitor and track your construction projects in one place.

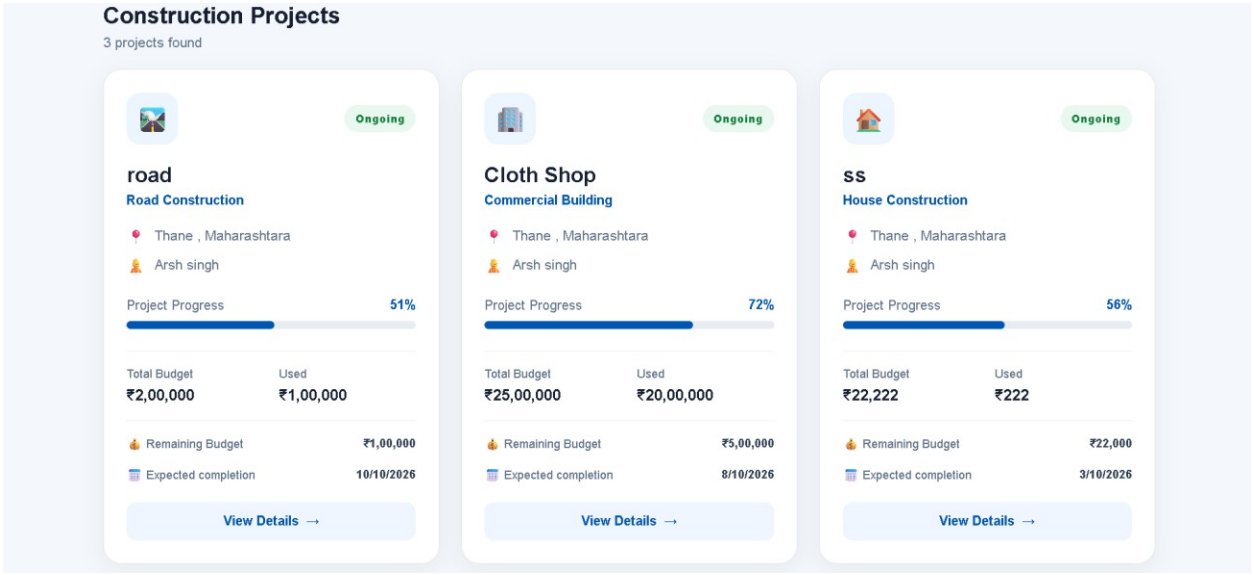
+ Add Project

Total Projects	Ongoing	Completed	Total Budget
3	3	0	₹27,22,222

Search projects, location or contractor...

All Ongoing Completed Planning






```
_id: ObjectId('6aace2bb870e3055a45ef43b')
projectName : "ss"
projectType : "House Construction"
location : "Thane , Maharashtara"
contractor : "Arsh singh"
budget : 22222
budgetUsed : 222
progress : 56
status : "Ongoing"
startDate : ISODate('2026-08-30T00:00:00.000+00:00') August 30, 2026 at 12:00:00 AM UTC
expectedCompletion : ISODate('2026-10-03T00:00:00.000+00:00') October 3, 2026 at 12:00:00 AM UTC
description : ""
createdAt : ISODate('2026-09-18T07:05:31.024+00:00') September 18, 2026 at 7:05:31 AM UTC
updatedAt : ISODate('2026-09-18T07:05:31.024+00:00') September 18, 2026 at 7:05:31 AM UTC
__v : 0
```

```
_id: ObjectId('6aace3cf870e3055a45ef43c')
projectName : "Cloth Shop"
projectType : "Commercial Building"
location : "Thane , Maharashtara"
contractor : "Arsh singh"
budget : 2500000
budgetUsed : 2000000
progress : 72
status : "Ongoing"
startDate : ISODate('2025-12-19T00:00:00.000+00:00') December 19, 2025 at 12:00:00 AM UTC
expectedCompletion : ISODate('2026-10-08T00:00:00.000+00:00') October 8, 2026 at 12:00:00 AM UTC
description : ""
createdAt : ISODate('2026-09-18T07:10:07.279+00:00') September 18, 2026 at 7:10:07 AM UTC
updatedAt : ISODate('2026-09-18T07:10:07.279+00:00') September 18, 2026 at 7:10:07 AM UTC
__v : 0
```

6aace3cf870e3055a45ef43c

```
_id: ObjectId('6aace9ca870e3055a45ef43d')
projectName : "road"
projectType : "Road Construction"
location : "Thane , Maharashtara"
contractor : "Arsh singh"
budget : 200000
budgetUsed : 100000
progress : 51
status : "Ongoing"
startDate : ISODate('2026-09-24T00:00:00.000+00:00') September 24, 2026 at 12:00:00 AM UTC
expectedCompletion : ISODate('2026-10-10T00:00:00.000+00:00') October 10, 2026 at 12:00:00 AM UTC
description : ""
createdAt : ISODate('2026-09-18T07:35:38.593+00:00') September 18, 2026 at 7:35:38 AM UTC
updatedAt : ISODate('2026-09-18T07:35:38.593+00:00') September 18, 2026 at 7:35:38 AM UTC
__v : 0
```